

CurrentUser Component Overview

Source Code

```
package CodeOrbit.backend.security;

import CodeOrbit.backend.exceptions.UnauthorizedException;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

@Component
public class CurrentUser {
    public AppUserPrincipal require() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth == null || !(auth.getPrincipal() instanceof AppUserPrincipal principal))
        {
            throw new UnauthorizedException("Not authenticated");
        }
        return principal;
    }
}
```

Explanation & Core Responsibilities

This component provides a clean utility method to extract and validate the authenticated user from Spring Security's context. Here is what it does step-by-step:

- **Fetches Security Context:** It inspects `SecurityContextHolder.getContext().getAuthentication()` to retrieve the active user's authentication details for the current request thread.
- **Validates Authentication:** It checks whether an authentication object exists and verifies that the principal is an instance of `AppUserPrincipal`.
- **Error Handling:** If the user is unauthenticated, anonymous, or missing required credentials, it throws a custom `UnauthorizedException("Not authenticated")`.
- **Returns Principal:** When valid, it directly returns the strongly typed `AppUserPrincipal`, enabling services and controllers to access the current user's profile and database ID cleanly without casting manually.

UserService Overview

Source Code

```
package CodeOrbit.backend.services;

import java.util.Map;
import java.util.UUID;

import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import CodeOrbit.backend.entity.User;
import CodeOrbit.backend.repository.UserRepository;

import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class UserService {
    public final UserRepository userRepository;
    public final TextEncryptor tokenEncryptor;
    // yahan user githubId se search hoga aur user entity banaaya jaaeyega aur DB me save ho'ga
    @Transactional
    public User upsertFromGitHub(Map<String, Object> attributes, String accessToken,
String scopes) {
        Long githubId = toLong(attributes.get("id"));
        String login = String.valueOf(attributes.get("login"));
        String name = attributes.get("name") != null
            ? String.valueOf(attributes.get("name"))
            : login;
        String avatarUrl = attributes.get("avatar_url") != null
            ? String.valueOf(attributes.get("avatar_url"))
            : null;

        String encryptedToken = tokenEncryptor.encrypt(accessToken);

        User user = userRepository.findByGithubId(githubId).orElseGet(User::new);
        user.setGithubId(githubId);
        user.setGithubUsername(login);
        user.setDisplayName(name);
        user.setAvatarUrl(avatarUrl);
        user.setAccessToken(encryptedToken);
        user.setTokenScopes(scopes);
    }
}
```

```

        return userRepository.save(user);
    }

    @Transactional(readOnly = true)
    public User requiredById(UUID id) {
        return userRepository.findById(id).orElseThrow(() -> new
IllegalArgumentException("User not found"));
    }

    public String decryptAccessToken(User user) {
        return tokenEncryptor.decrypt(user.getAccessToken());
    }

    private static Long toLong(Object value) {
        if (value instanceof Number number) {
            return number.longValue();
        }
        return Long.parseLong(String.valueOf(value));
    }
}

```

Explanation & Core Responsibilities

This class manages database persistence and encryption logic for user accounts synchronized with GitHub. Here are its main functions:

- **GitHub User Upserting:** Extracts the user's GitHub ID, username, display name, and avatar URL. It searches the database for an existing user by `githubId`; if found, it updates the record, otherwise, it creates a new `User` instance.
- **Token Encryption:** Uses `TextEncryptor` to securely encrypt GitHub OAuth2 access tokens before saving them into the database, protecting raw credentials at rest.
- **Token Decryption:** Provides `decryptAccessToken(User user)` to retrieve and decrypt the raw GitHub token whenever the application needs to make authorized API calls on behalf of the user.
- **User Retrieval & Helper Logic:** Includes `requiredById(UUID id)` to securely fetch a user or throw an exception if not found, as well as `toLong()` to safely parse GitHub user IDs from generic objects.

GithubOAuth2UserService Overview

Source Code

```
package CodeOrbit.backend.security;

import org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
import org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
import org.springframework.security.oauth2.client.userinfo.OAuth2UserService;
import org.springframework.security.oauth2.core.OAuth2AuthenticationException;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.stereotype.Service;

import CodeOrbit.backend.entity.User;
import CodeOrbit.backend.services.UserService;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class GithubOAuth2UserService implements OAuth2UserService<OAuth2UserRequest,
OAuth2User> {
    private final UserService userService;
    private final DefaultOAuth2UserService delegate = new DefaultOAuth2UserService();

    @Override
    public OAuth2User loadUser(OAuth2UserRequest userRequest) throws
    OAuth2AuthenticationException {
        OAuth2User githubUser = delegate.loadUser(userRequest);
        // ye github se crrentlogin user ko laayega

        String accessToken = userRequest.getAccessToken().getTokenValue();
        String scopes = userRequest.getAccessToken().getScopes() != null
            ? String.join(",", userRequest.getAccessToken().getScopes())
            : "read:user,repo";

        User user = userService.upsertFromGitHub(githubUser.getAttributes() ,
accessToken , scopes);
        return new AppUserPrincipal(user , githubUser.getAttributes());
    }
}
```

Explanation & Execution Flow

This class follows the standard Spring Security OAuth2 service pattern to process user logins from GitHub. Here is what happens step-by-step when a user logs in:

- **Delegates to Spring Security:** It uses `DefaultOAuth2UserService` to fetch the user profile attributes directly from GitHub's `/user` endpoint using the OAuth2 token.
- **Extracts Credentials & Scopes:** It extracts the OAuth2 `accessToken` string and the granted OAuth2 scopes (e.g., `read:user`, `repo`).
- **Saves/Updates User:** It passes the attributes, access token, and scopes to `userService.upsertFromGitHub(...)`. This creates a new user in your database if they don't exist, or updates their details and access token if they already do.
- **Returns Custom Principal:** It wraps the database `User` object along with the raw GitHub attributes inside `AppUserPrincipal` (which implements `OAuth2User`), allowing you to access custom domain user data directly in Spring Security contexts (e.g., via `@AuthenticationPrincipal`).

CodeOrbit / backend / security / AppUserPrincipal.java

OAuth2 Principal Implementation Source Code

```
package CodeOrbit.backend.security;

import java.util.Collection;
import java.util.Map;
import java.util.UUID;

import CodeOrbit.backend.entity.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.AuthorityUtils;
import org.springframework.security.oauth2.core.user.OAuth2User;

// AppUserPrincipal ek Spring Security OAuth2 user hai.
public class AppUserPrincipal implements OAuth2User {

    private final User user;
    private final Map<String, Object> attributes;

    public AppUserPrincipal(User user, Map<String, Object> attributes) {
        this.user = user;
        this.attributes = attributes;
    }

    public UUID getId() {
        return user.getId();
    }

    public User getUser() {
        return user;
    }

    @Override
    public Map<String, Object> getAttributes() {
        return attributes;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return AuthorityUtils.createAuthorityList("ROLE_USER");
    }

    @Override
    public String getName() {
        return user.getId().toString();
    }
}
```

CodeOrbit / backend / controllers / AuthController.java

Authentication Controller Source Code

```
package CodeOrbit.backend.controllers;

import CodeOrbit.backend.dto.UserResponse;
import CodeOrbit.backend.entity.User;
import CodeOrbit.backend.security.AppUserPrincipal;
import CodeOrbit.backend.security.CurrentUser;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpSession;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final CurrentUser currentUser;

    @GetMapping("/login-url")
    public Map<String, String> loginUrl() {
        return Map.of(
            "url",
            "/oauth2/authorization/github"
        );
    }

    @GetMapping("/me")
    public ResponseEntity<UserResponse> me() {
        AppUserPrincipal principal = currentUser.require();
        User user = principal.getUser();

        return ResponseEntity.ok(
            new UserResponse(
                user.getId(),
                user.getGithubId(),
                user.getGithubUsername(),
                user.getDisplayName(),
                user.getAvatarUrl()
            )
        );
    }

    @PostMapping("/logout")
```

```
public ResponseEntity<Void> logout(  
    HttpServletRequest request  
) {  
    System.out.println("logging out");  
  
    HttpSession session = request.getSession(false);  
  
    if (session != null) {  
        session.invalidate();  
    }  
  
    return ResponseEntity.noContent().build();  
}  
}
```

CodeOrbit / backend / config / CorsConfig.java

CORS Security Configuration Source Code

```
package CodeOrbit.backend.config;

import java.util.Arrays;
import java.util.List;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
public class CorsConfig {

    @Bean
    CorsConfigurationSource corsConfigurationSource(
        @Value("${app.cors.allowed-origins}") String allowedOrigins) {
        CorsConfiguration config = new CorsConfiguration();
        List<String> origins = Arrays.stream(allowedOrigins.split(","))
            .map(String::trim)
            .filter(s -> !s.isEmpty())
            .toList();
        config.setAllowedOrigins(origins);
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE",
"OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);
        config.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        // backend ke saare URL paths , sab par ye config apply hoga.
        return source;
    }
}
```

CodeOrbit / backend / config / SecurityConfig.java

Spring Security Configuration Source Code

```
package CodeOrbit.backend.config;

import CodeOrbit.backend.security.GithubOAuth2UserService;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.http.HttpStatus;

import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.AuthenticationFailureHandler;
import org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.SimpleUrlAuthenticationFailureHandler;
import org.springframework.security.web.authentication.SimpleUrlAuthenticationSuccessHandler;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(
        HttpSecurity http,
        GithubOAuth2UserService githubOAuth2UserService,
        AuthenticationSuccessHandler oauth2SuccessHandler,
        AuthenticationFailureHandler oauth2FailureHandler
    ) throws Exception {

        http

            // Use CorsConfig.java for CORS configuration
            .cors(Customizer.withDefaults())

            // CSRF disabled for now
            .csrf(csrf -> csrf.disable())

            // Session is created when required
            .sessionManagement(session -> session
                .sessionCreationPolicy(
```

```

        SessionCreationPolicy.IF_REQUIRED
    )
)

.authorizeHttpRequests(auth -> auth

    // OAuth endpoints must be public
    .requestMatchers(
        "/api/auth/login-url",
        "/oauth2/**",
        "/login/oauth2/**",
        "/error"
    ).permitAll()

    // CORS preflight requests
    .requestMatchers(
        HttpMethod.OPTIONS,
        "/"**"
    ).permitAll()

    // All API endpoints require authentication
    .requestMatchers("/api/**")
    .authenticated()

    // Everything else can be accessed publicly
    .anyRequest()
    .permitAll()
)

// Return 401 instead of redirecting to login
// for unauthenticated API requests
.exceptionHandling(ex -> ex
    .authenticationEntryPoint(
        new HttpStatusEntryPoint(
            HttpStatus.UNAUTHORIZED
        )
    )
)

// GitHub OAuth2
.oauth2Login(oauth -> oauth

    .userInfoEndpoint(userInfo -> userInfo
        .userService(
            githubOAuth2UserService
        )
    )

    .successHandler(
        oauth2SuccessHandler
    )
)

```

```

        )

        .failureHandler(
            oauth2FailureHandler
        )
    )

    // Logout
    .logout(logout -> logout

        .logoutUrl("/api/auth/logout")

        .logoutSuccessHandler(
            (request, response, authentication) ->
                response.setStatus(
                    HttpStatus.NO_CONTENT.value()
                )
        )

        .invalidateHttpSession(true)

        .clearAuthentication(true)

        .deleteCookies("CODEORBIT_SESSION")
    );

    return http.build();
}

// =====
// OAUTH SUCCESS
// =====

@Bean
AuthenticationSuccessHandler oauth2SuccessHandler(
    @Value("${app.frontend-url}") String frontendUrl
) {

    SimpleUrlAuthenticationSuccessHandler handler =
        new SimpleUrlAuthenticationSuccessHandler();

    handler.setDefaultTargetUrl(
        frontendUrl + "/auth/callback"
    );

    return handler;
}

```

```
// =====  
// OAUTH FAILURE  
// =====  
  
@Bean  
AuthenticationFailureHandler oauth2FailureHandler(  
    @Value("${app.frontend-url}") String frontendUrl  
) {  
  
    SimpleUrlAuthenticationFailureHandler handler =  
        new SimpleUrlAuthenticationFailureHandler();  
  
    handler.setDefaultFailureUrl(  
        frontendUrl + "/login?error=oauth_failed"  
    );  
  
    return handler;  
}  
}
```